

Loan Calculator Project Report

Project Name: Loan Calculator Application

Group Members : Muhammad Saqlain Khan (24P-3047), Muhammad Haris Malik (24P-3059)

Section: BSE 3A

1. Project Overview

The **Loan Calculator** is an application designed to compute financial values related to loans. It supports both **Graphical User Interface (GUI)** and **Command Line Interface (CLI)** modes. The application allows users to calculate:

- Monthly payment (EMI)
- Total payment
- Remaining balance
- Number of payments
- Interest rate
- Loan amount

The project emphasizes **accuracy, flexibility, and maintainability**, with additional support for configuration files and unit testing.

2. Code snippets of bug fixes and refactorings Bugs and Fixes

Bug 1: Negative Input Values

- **Issue:** Negative loan amount, interest, or periods caused division by zero, logarithm of negative numbers, and undesired results.
- **Fix:** Added input validation inside setter functions.

```

inline void setAmount(float A) {
    if(A <= 0) throw std::invalid_argument("Loan amount must be > 0");

} inline void setInterest(float i) {    if(i <= 0) throw

std::invalid_argument("Interest must be > 0");

}

inline void setPeriodTotal(int N) {    if(N <= 0) throw

std::invalid_argument("Total period must be > 0 months");

}

```

Bug 2: Precision Overflow for Large Tenure

- **Issue:** Calculating $(1+i)^N$ for large N (e.g., 360 months) caused float overflow.
- **Fix:** Changed all financial calculations to `long double` to maintain precision.

```

long double i = (long double)interestPeriodic_; long double
totalAmount = (long double)amount_; long double denom = (1 -
pow((1+i), (-1*(long double)periodTotal_))); long double result = (i *
totalAmount) / denom;

```

Bug 3: Hardcoded Defaults

- **Issue:** Opening fee, interest, and percentages were hard-coded.
- **Fix:** Added support for reading defaults from a configuration file ifstream

```

f("config.txt"); while(f >> key >> val){    if(key == "default_interest")
calculator.setInterest(val);    else if(key == "default_opening_fee")
calculator.setOpeningFee(val);

}

```

Bug 4: Invalid Logarithm in Payment Calculation

- **Issue:** If the monthly payment was too small, $1 - i*A/P \leq 0$ caused domain errors.

- **Fix:** Added validation before calculating logarithms.

`long double arg = 1.0L - (i * A / P); if(!(arg > 0.0L)) throw`

`std::invalid_argument("Payment is too small.");`

Refactorings:-

Main.cpp

```
/**
 * @file main.cpp
 * @brief Contains the reference implementation for Loan calculations.
 *
 * This file demonstrates the standard, correct implementation of the
 * Loan::monthlyPayment method, intended as the final application logic.
 */
#include <cmath>
#include "Loan.h" // Include the header file

/**
 * @brief Calculates the Equal Monthly Installment (EMI) based on the
 * standard amortization formula.
 *
 * This version serves as the canonical implementation. It handles the 0%
 * rate case
 *
 * but assumes the inputs are reasonable and does not include the explicit
 * LargeTenure * overflow check, relying on standard
 * library behavior.
```

```

*
* @return The monthly payment amount (EMI), rounded to two decimal places.
*/

long double Loan::monthlyPayment() const {
    long double monthlyRate = rate / 12.0 / 100.0;

    int totalMonths = years * 12;

    long double pow_term = pow(1.0 + monthlyRate, totalMonths); // Used 1.0
to match convention

    long double emi;

    if (monthlyRate ==
0) {

        // Simple principal division for 0% rate

        emi = amount / totalMonths;

    } else {

        // Standard EMI calculation

        long double numerator = amount * monthlyRate * pow_term;

        long double denominator = pow_term - 1.0; // Used 1.0 to match
convention

        emi = numerator / denominator;

    }

    // Round to 2 decimal places

```

```
        return std::round(emi * 100.0) / 100.0;
    }
}
```

Loan.cpp

```
/**
 * @file Loan.cpp
 * @brief Implementation of the Loan class methods.
 *
 * This file contains the implementation details for loan calculations,
 * including validation, monthly payment, total payment, and total
 * interest.
 *
 * It includes robust handling for high-exponent scenarios (LargeTenure
 * fix).
 */

#include "Loan.h"    // Include the class definition

#include <cmath>      // Include for std::pow, std::round, and
std::isfinite

/**
 * @brief Constructor for the Loan class.
 *
 * @param a The principal amount of the loan.
 *
 * @param r The annual interest rate (percentage).
 *
 * @param y The term of the loan in years.
 */
Loan::Loan(long double a, long double r, int y) : amount(a), rate(r),
```

```
years(y) {}
```

```
/**
 * @brief Checks if the loan parameters are valid.
 * @return true if amount > 0, rate >= 0, and years > 0.
 */
bool Loan::isValid()
const {

    return amount > 0 && rate >= 0 && years > 0;
}

/**
 * @brief Calculates the Equal Monthly Installment (EMI).
 *
 * Formula:  $EMI = P * r * (1+r)^n / ((1+r)^n - 1)$ 
 *
 * Includes a special case for 0% interest and a check for floating-point
overflow
 * in the exponentiation term (pow_term).
 *
 * @return The monthly payment amount (EMI), rounded to two decimal places.
 */
long double Loan::monthlyPayment()
const {

    // Convert annual percentage rate to monthly decimal rate
```

```
long double monthlyRate = rate / 12.0 / 100.0;
```

```
int totalMonths = years * 12;
```

```
long double emi;
```

```
if (monthlyRate == 0) {
```

```
    // Special case: 0% interest (simple division)
```

```
    emi = amount / totalMonths;
```

```
} else {
```

```
    // Standard EMI calculation logic
```

```
    long double pow_term = std::pow(1.0 + monthlyRate, totalMonths);
```

```
    // FIX for LargeTenure: Check if the exponent term has overflowed  
    to infinity.
```

```
    if (!std::isfinite(pow_term)) { // If  $(1+r)^n$   
        overflows, the EMI simplifies to  $P * r$  (Interest
```

```
only in perpetuity).
```

```
    // For extremely long terms, the principal component of payment  
    approaches zero.
```

```
    emi = amount * monthlyRate;
```

```
    } else {
```

```
        // Standard EMI calculation long double  
        numerator = amount * monthlyRate * pow_term;
```

```

        long double denominator = pow_term - 1.0;

        emi = numerator / denominator;

    }

}

// Round the EMI to 2 decimal places for financial accuracy

return std::round(emi * 100.0) / 100.0;
}

/**
 * @brief Calculates the total payment made over the lifetime of the loan.
 * @return The total principal and interest paid.
 */

long double Loan::totalPayment() const {

    return monthlyPayment() * years * 12;
}

/**
 * @brief Calculates the total interest paid over the lifetime of the loan.
 * @return The total interest paid (Total Payment - Principal Amount).
 */

long double Loan::totalInterest() const {

```



```
return totalPayment() - amount; }
```

Loan.h

```
/**
```

```
* @file Loan.h
```

```
* @brief Defines the Loan class for calculating loan metrics (EMI, total  
payment, interest).
```

```
*
```

```
* This header defines the structure and public interface for the Loan  
class,
```

```
* which is used to model a simple amortizing loan.
```

```
*/
```

```
#ifndef LOAN_H
```

```
#define LOAN_H
```

```
/**
```

```
* @class Loan
```

```
* @brief Represents a single fixed-rate, amortizing loan.
```

```
*
```

```
* The class holds the principal amount, annual interest rate, and term in  
years,
```

```
* and provides methods to calculate key financial figures like monthly
```

```
payment. */

class Loan {

private:

    /// The principal amount of the loan (long double for high precision).

    long double amount;

    /// The annual interest rate in percent (e.g., 5.0 for 5%).

    long double rate;
```

```
    /// The term of the loan in full years.

    int years;

public:
/**

    * @brief Constructor for the Loan class.

    * @param a The principal amount of the loan.

    * @param r The annual interest rate (percentage).

    * @param y The term of the loan in years.

    */

    Loan(long double a, long double r, int y);

/**

    * @brief Checks if the loan parameters are valid.
```

```

    * @return true if amount > 0, rate >= 0, and years > 0.

    */

bool isValid() const;

/**

    * @brief Calculates the Equal Monthly Installment (EMI).

    *

    * The calculation uses the standard amortization formula and handles
the edge case

```

```

    * of extremely long tenures to prevent floating-point overflow. The
result is

    * rounded to two decimal places.

    *      * @return The monthly payment amount
(EMI).

    */

long double monthlyPayment() const;

/**

    * @brief Calculates the total payment made over the lifetime of the
loan.      * @return The total principal and interest
paid.

    */

long double totalPayment() const;

```

```

    /**
     * @brief Calculates the total interest paid over the lifetime of the
    loan.      * @return The total interest paid (Total Payment - Principal
    Amount) .

    */

    long double totalInterest() const;
};

#endif

```

Overall many refactorings is done in the code including all files (main.cpp, Loan.h, loan.cpp)

Test Output:

The project uses **GoogleTest** to verify correctness:

- **Normal EMI Test:** Checks if the monthly payment is calculated correctly.
- **Invalid Input Test:** Verifies that negative or zero values throw exceptions.
- **Large Tenure Test:** Ensures calculations remain finite for long-term loans.

```

● msaqlainkhan@msaqlainkhan:~/legacy-calc-2009$ # 1. Recompile the corrected Loan.cpp
g++ -std=c++17 -c src/Loan.cpp -o Loan.o

# 2. Recompile and link the tests
g++ -std=c++17 test/test_emi.cpp Loan.o -lgtest -lgtest_main -pthread -o runTests

# 3. Execute the tests
./runTests
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanTest
[ RUN      ] LoanTest.NormalEMI
[       OK ] LoanTest.NormalEMI (0 ms)
[ RUN      ] LoanTest.InvalidInput
[       OK ] LoanTest.InvalidInput (0 ms)
[ RUN      ] LoanTest.LargeTenure
[       OK ] LoanTest.LargeTenure (0 ms)
[-----] 3 tests from LoanTest (0 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (0 ms total)
[ PASSED  ] 3 tests.

```

Git commit history:

Commits

dev

All users

All time

Commits on Nov 26, 2025

Add test cases and Doxygen file

p243047 committed 3 minutes ago

e927e74

Commits on Nov 25, 2025

Fixing Bugs, make loan.h and loan.cpp

p243047 committed 1 hour ago

4ddda3a

inital changes and setup

p243047 committed 3 hours ago

8467a19

chore: add .gitignore

p243047 committed 4 hours ago

2165cdd

test cases, doxygen output and file

Verified

da51124



1

Haris718 authored 1 minute ago

 Commits on Nov 25, 2025

initial values to config.txt

Verified

bb92385



1

Haris718 authored 2 hours ago

created main.cpp for loans

Verified

dc34990



1

Haris718 authored 2 hours ago

implemented loan class

Verified

1163469



1

Haris718 authored 2 hours ago

Create loan.h

Verified

9d3149b



1

Haris718 authored 2 hours ago

Delete makefile

Verified

539c7a4



1

Haris718 authored 3 hours ago

Delete SConstruct

Verified

224af5e



1

Haris718 authored 3 hours ago

Delete LoanCalculatorMain.cpp

Verified

95b7089



1

Haris718 authored 3 hours ago

Links:

<https://github.com/Haris718/legacy-calc-2009/tree/dev>

<https://github.com/p243047/legacy-calc-2009/tree/dev>